

🦘 OzZoo Design Document – Student Implementation Guide

What You're Building

You're creating **OzZoo**, a text-based wildlife park simulation game where you manage an Australian zoo. Players feed animals, clean enclosures, manage money, and try to keep visitors happy while maintaining animal welfare.

🎯 The Big Picture

Think of this as a mini-game with three main responsibilities:

1. **Animals** need care (feeding, health monitoring)
2. **Enclosures** need maintenance (cleaning, capacity limits)
3. **Zoo** needs management (budget, visitors, daily operations)

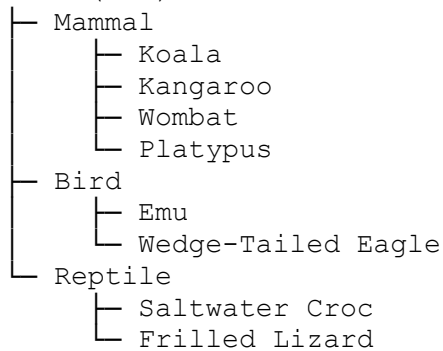
Each day, the player makes decisions, then the day advances and consequences happen (hunger increases, health changes, revenue is calculated).

🧱 Core OOP Concepts You'll Demonstrate

1. Inheritance Hierarchy (3 Levels)

...

Animal (ABC)



...

- **Level 1 (Animal):** Abstract base with shared attributes (name, health, hunger, happiness)
- **Level 2 (Mammal/Bird/Reptile):** Adds category-specific features (fur_color, wingspan, is_venomous)
- **Level 3 (Koala/Emu/Croc):** Concrete implementations with unique behaviors

2. Polymorphism

Every animal implements `make_sound()` differently:

- Koala: "deep bellowing sound"
- Kangaroo: "clucking sound"
- Emu: "booming call"

Same method name, different behavior = ****polymorphism****.

3. Encapsulation

Use ****protected**** (``_attribute``) and ****private**** (``__attribute``) naming:

```
```python
self._health = 100 # Protected
self.__secret = "data" # Private
```
```

Provide controlled access via ****properties****:

```
```python
@property
def health(self):
 return self._health

@health.setter
def health(self, value):
 self._health = max(0, min(100, value)) # Keep in range 0-100
```
```

4. Abstract Base Classes (ABCs)

``Animal`` is ****abstract**** – you can't create an ``Animal`` directly; you must create a ``Koala`` or ``Kangaroo``.

Mark required methods with ``@abstractmethod``:

```
```python
@abstractmethod
def make_sound(self) -> str:
 pass
```
```

Every subclass ****must**** implement this method.

5. Interfaces

``ICleanable`` and ``IFeedable`` define contracts:

- ****ICleanable**** "Any class using this must have ``clean()`` and ``get_cleanliness()`` methods"
- ****IFeedable**** "Any class using this must have ``feed()``, ``get_diet()``, and ``is_hungry()`` methods"

Example:

```
```python
class Animal(ABC, IFeedable): # Animal MUST implement IFeedable methods
 def feed(self, food):
 # Implementation here
```
```

6. Design Pattern: Factory

Instead of writing:

```
```python
koala = Koala(name="Blinky", age=3)
kangaroo = Kangaroo(name="Skippy", age=5)
```
```

You write:

```
```python
factory = AnimalFactory()
koala = factory.create_animal("koala", name="Blinky", age=3)
kangaroo = factory.create_animal("kangaroo", name="Skippy", age=5)
```
```

****Why?**** Centralizes creation logic, makes it easy to add new animals, and validates input in one place.

7. Custom Exceptions

Create a hierarchy of exceptions:

```
...
ZooError
├── AnimalError
│   ├── IncompatibleSpeciesError
│   └── AnimalSickError
├── ResourceError
│   ├── InsufficientFoodError
│   └── InsufficientFundsError
└── EnclosureError
    ├── HabitatCapacityExceededError
    └── InvalidHabitatError
...
```

Use these to handle specific error conditions gracefully:

```
```python
try:
 zoo.add_animal(koala, enclosure)
except HabitatCapacityExceededError as e:
 print(f"Error: {e.message}")
...

```

## ## 🎮 Game Mechanics Explained

### ### Daily Cycle

Each day follows this pattern:

1. **\*\*Morning:\*\*** Display report, hunger increases (+15 for all animals)
2. **\*\*Player Actions:\*\*** Feed, clean, buy, sell, etc.
3. **\*\*Evening:\*\***
  - Calculate revenue (ticket sales + donations)
  - Unfed animals lose health (-5)
  - Dirty enclosures reduce happiness (-10)
  - Check for breeding opportunities
  - Advance to next day

### ### Animal Needs

Every animal has three key stats (0-100 scale):

Stat	What it means	What happens if low
--- ----- -----		
**Health**	Overall wellness	< 20 = sick, = 0 = dies
**Hunger**	How hungry they are	> 80 = starving, lose health faster
**Happiness**	Mood/contentment	< 30 = stressed, won't breed

### ### Breeding System

Animals can produce offspring if:

1. Two of the same species
2. Both health > 70
3. Both happiness > 60
4. Enclosure has space
5. 10% chance per day

---

### ## 📁 File Structure (Simplified)

For your submission, you can use:

```
...
ozzoo_game/
├── ozzoo_main.py # Game loop, entry point
├── ozzoo_models.py # All classes (Animal, Zoo, Enclosure, etc.)
└── ozzoo_ui.py # CLI interface
...
```

Or consolidate everything into **one file** if preferred (easier for grading).

---

### ## ✅ Implementation Checklist

#### ### Must Have (Core Requirements)

- ✅ 10+ classes (you have 15+)
- ✅ Encapsulation (`\_protected`, `\_\_private`)
- ✅ 3-level inheritance (Animal → Mammal → Koala)
- ✅ Polymorphism (`make\_sound()` overrides)
- ✅ 1+ ABC (`Animal`, `Enclosure`)
- ✅ 1+ Interface (`ICleanable`, `IFeedable`)
- ✅ 1+ Design Pattern (`AnimalFactory`)
- ✅ 1+ Custom Exception (`ZooError` hierarchy)
- ✅ Game loop (day-by-day simulation)
- ✅ CLI interface
- ✅ Resource management (budget, food stock)

#### ### Should Have (for higher marks)

- Breeding mechanic
- Multiple enclosure types (EucalyptusGrove, OutbackSavanna, Billabong, etc.)
- Random events (heatwave, VIP visitor, escape attempt)
- Visitor satisfaction tracking

---

## ## 💡 Key Implementation Tips

### ### 1. Start with the hierarchy

Build `Animal` → `Mammal` → `Koala` first, test it, then add more animals.

### ### 2. Use properties everywhere

Control access to attributes:

```
```python
@property
def health(self):
    return self._health
```
```

### ### 3. Test incrementally

Don't write everything at once. Build one class, test it in isolation, then move on.

### ### 4. Keep the CLI simple

Use `input()` and `print()`. A fancy UI isn't required.

### ### 5. Don't overcomplicate

You don't need save/load or a GUI to get a great mark. Focus on demonstrating OOP principles correctly.

### ### 6. Comment wisely

Explain **why**, not **what**:

```
```python
# Good
self._hunger += 15 # Daily metabolism increases hunger

# Not helpful
self._hunger += 15 # Add 15 to hunger
```
```

---

## ## 🎨 Example Output

...

|                       |                 |
|-----------------------|-----------------|
| 🦘 Welcome to OZZOO! 🦘 |                 |
| Day: 3                | Budget: \$8,450 |
| Visitors Today: 89    |                 |
| [1] View Animals      |                 |
| [2] Feed Animals      |                 |

```
[3] Clean Enclosures
[9] Advance Day
```

Enter choice: 2

Which animal?

1.  Blinky (Koala) - Health: 85, Hunger: 65
  2.  Skippy (Kangaroo) - Health: 55, Hunger: 90
- > 2

```
✓ Skippy happily munches on grass!
 Hunger: 90 → 40
 Happiness: 50 → 55
...

```

##  Class Structure Reference

### Animal Hierarchy

```
```python
class Animal(ABC, IFeedable):
    """Abstract base class for all animals"""

    def __init__(self, name: str, age: int, species: str):
        self._name = name
        self._age = age
        self._health = 100
        self._hunger = 30
        self._happiness = 70

    @abstractmethod
    def make_sound(self) -> str:
        pass

    @abstractmethod
    def get_diet(self) -> str:
        pass

    @abstractmethod
    def get_habitat_type(self) -> str:
        pass

    def feed(self, food: 'Food') -> bool:
        if food.food_type == self.get_diet():
            self._hunger = max(0, self._hunger - food.nutrition)
            return True
        return False
...

```

Enclosure Hierarchy

```
```python
class Enclosure(ABC, ICleanable):
 """Abstract base class for enclosures"""

```

```

def __init__(self, enclosure_id: str, name: str, capacity: int):
 self._enclosure_id = enclosure_id
 self._name = name
 self._capacity = capacity
 self._animals = []
 self._cleanliness = 100

 @abstractmethod
 def clean(self) -> None:
 pass

 def add_animal(self, animal: Animal) -> None:
 if len(self._animals) >= self._capacity:
 raise HabitatCapacityExceededError(self._name,
self._capacity)
 self._animals.append(animal)
...

```

### Zoo Class

```

```python
class Zoo:
    """Main zoo management class"""

    def __init__(self, name: str, starting_budget: float):
        self._name = name
        self._budget = starting_budget
        self._enclosures = []
        self._animals = []
        self._visitors = []
        self._day = 1
        self._food_stock = {}

    def advance_day(self) -> None:
        """Progress to the next day"""
        # Update all animals
        for animal in self._animals:
            animal.daily_update()

        # Calculate revenue
        self._calculate_revenue()

        # Increment day counter
        self._day += 1
...

```

AnimalFactory Pattern

```

```python
class AnimalFactory:
 """Factory for creating animals"""

 _animal_types = {
 "koala": Koala,
 "kangaroo": Kangaroo,
 "emu": Emu,
 # ... more types
 }

```

```

 @classmethod
 def create_animal(cls, animal_type: str, **kwargs) -> Animal:
 animal_type = animal_type.lower().replace(" ", "_")

 if animal_type not in cls._animal_types:
 raise ValueError(f"Unknown animal type: {animal_type}")

 animal_class = cls._animal_types[animal_type]
 return animal_class(**kwargs)
...

🎯 Project Requirements Summary

OOP Principles to Demonstrate

1. Encapsulation: Private/protected attributes with property accessors
2. Inheritance: 3-level hierarchy (Animal → Mammal → Koala)
3. Polymorphism: Method overriding (`make_sound()`, `get_diet()`)
4. Abstraction: Abstract base classes that cannot be instantiated
5. Composition: Zoo has-a collection of Animals and Enclosures

Design Elements Required

1. Abstract Base Classes (ABCs): At least 1 (`Animal`, `Enclosure`)
2. Interfaces: At least 1 (`ICleanable`, `IFeedable`)
3. Design Patterns: At least 1 (Factory pattern for animal creation)
4. Exception Handling: Custom exception hierarchy
5. Type Hints: Use throughout for clarity

Functional Requirements

1. Game Loop: Day-by-day simulation
2. Resource Management: Budget, food stock tracking
3. Animal Care System: Health, hunger, happiness mechanics
4. Enclosure Management: Cleaning, capacity constraints
5. CLI Interface: Menu-driven interaction

📋 Assessment Alignment

This project demonstrates:

- ✅ Advanced OOP concepts: ABCs, interfaces, multiple inheritance
- ✅ Design patterns: Factory pattern for object creation
- ✅ Code organisation: Clear separation of concerns
- ✅ Error handling: Custom exception hierarchy
- ✅ Documentation: Type hints, docstrings, comments
- ✅ Testing capability: Modular design allows unit testing

```

## ## 🚀 Getting Started Steps

1. **\*\*Set up your project structure\*\*** (create folders/files)
2. **\*\*Define exceptions\*\*** (`zoo\_exceptions.py`)
3. **\*\*Create interfaces\*\*** (`ICleanable`, `IFeedable`)
4. **\*\*Build Animal hierarchy\*\***:
  - Start with `Animal` ABC
  - Add `Mammal`, `Bird`, `Reptile`
  - Implement `Koala`, `Kangaroo` (test these first)
  - Add remaining animals
5. **\*\*Build Enclosure hierarchy\*\***:
  - Create `Enclosure` ABC
  - Implement `EucalyptusGrove`, `OutbackSavanna`
6. **\*\*Create supporting classes\*\***:
  - `Food`
  - `Medicine`
  - `Visitor`
  - `ZooKeeper`
7. **\*\*Implement AnimalFactory\*\***
8. **\*\*Build Zoo class\*\*** (main game controller)
9. **\*\*Create CLI interface\*\***
10. **\*\*Test everything together\*\***
11. **\*\*Add polish\*\*** (random events, breeding, better formatting)

---

## ## 🌟 Bottom Line

This is a well-structured OOP project that demonstrates inheritance, polymorphism, encapsulation, ABCs, interfaces, design patterns, and exception handling through a fun simulation game.

**\*\*Start with the core hierarchy, build incrementally, and test as you go.\*\***

You've got this! 🚀

---

\*Implementation Guide for NIT2112 Object-Oriented Programming\*  
\*OzZoo - Australian Wildlife Park Simulation\*  
\*Victoria University\*